

Stack Overflow is a question and answer site for professional and enthusiast programmers. It's 100% free, no registration required.

[Take the 2-minute tour](#)


Approximate substring matching using a Suffix Tree

`$ git push origin stackoverflowcareers` [Add repos](#)

This article discusses approximate substring matching techniques that utilize a [suffix tree](#) to improve matching time. Each answer addresses a different algorithm.

1. Approximate substring matching attempts to find a substring (pattern) P in a string T allowing up to k mismatches.
2. To learn how to create a suffix tree, click [here](#). However, some algorithms require additional preprocessing.

I invite people to add new algorithms (even if it's incomplete) and improve answers.

[algorithm](#) [string-matching](#) [suffix-tree](#) [edit-distance](#)

edited Oct 26 '13 at 10:00

community wiki
14 revs
Pooven

I added what I can figure out. I'm out of time. Good luck with it! – [Gene](#) Oct 27 '13 at 20:00

2 Answers

You are doing well. I don't have familiarity with the algorithm, but have read the paper today. Everything you wrote is correct as far as it goes. You are right that some parts of the explanation assume a lot.

Your Questions

1. What does size of the output refer to in terms of the suffix tree or input strings? The final output phase lists all occurrences of $\text{Key}(r)$ in T , for all states r marked for output.

The output consists of the maximal k -distance matches of P in T . In particular you'll get the final index and length for each. So clearly this is also $O(n)$ (remember big- O is an upper bound), but may be smaller. This is a nod to the fact that it's impossible to generate p matches in less than $O(p)$ time. The rest of the time bound concerns only the pattern length and the number of viable prefixes, both of which can be arbitrarily small, so the output size can dominate. Consider $k=0$ and the input is 'a' repeated n times with the pattern 'a'.

2. Looking at Algorithm C, the function dp is defined (page four); I don't understand what index i represents. It isn't initialized and doesn't appear to increment.

You're right. It's an error. The loop index should be i . What about j ? This is the index of the column corresponding to the input character being processed in the dynamic program. It should really be an input parameter.

Let's take a step back. The **Example** table on page 6 is computed left-to-right, column-by-column using equations (1-4) given earlier. These show that only the previous columns of D and L are needed to get the next. Function dp is just an implementation of this idea of computing column j from $j-1$. Column j of D and L are called d and l respectively. Column $j-1$ D and L are d' and l' , the function input parameters.

I recommend you work through the dynamic program until you understand it well. The algorithm is all about avoiding duplicate column computations. Here "duplicate" means "having the same values in the *essential* part", because that's all that matters. The inessential parts can't affect the answer.

The uncompressed trie is just the compressed one expanded in the obvious way to have one edge per character. Except for the idea of "depth", this is unimportant. In the compressed tree, $\text{depth}(s)$ is just the length of the string - which he calls $\text{Key}(s)$ - needed to get from root node s .

Algorithm A

Algorithm A is just a clever caching scheme.

All his theorems and lemmas show that 1) we only need to worry about the *essential* parts of

columns and 2) the essential part of a column j is completely determined by the viable prefix Q_j . This is the longest suffix of the input ending at j that matches a prefix of the pattern (within edit distance k). In other words, Q_j is the maximal start of a k -edit match at the end of the input considered so far.

With this, here's pseudo-code for Algorithm A.

```

Let r = root of (uncompressed) suffix trie
Set r's cached d,l with formulas at end page 7 (0'th dp table columns)
// Invariant: r contains cached d,l
for each character t_j from input text T in sequence
  Let s = g(r, t_j) // make the go-to transition from r on t_j
  if visited(s)
    r = s
    while no cached d,l on node r
      r = f(r) // traverse suffix edge
    end while
  else
    Use cached d',l' on r to find new columns (d,l) = dp(d',l')
    Compute |Q_j| = l[h], h = argmax(i).d[i]<=k as in the paper
    r = s
    while depth(r) != |Q_j|
      mark r visited
      r = f(r) // traverse suffix edge
    end while
    mark r visited
    set cached d,l on node r
  end if
end for

```

I've left out the output step for simplicity.

What is traversing suffix edges about? When we do this from a node r where $\text{Key}(r) = aX$ (leading a followed by some string X), we are going to the node with $\text{Key } X$. The consequence: we are storing each column corresponding to a viable prefix Q_h at the trie node for the suffix of the input with prefix Q_h . The function $f(s) = r$ is the suffix transition function.

For what it's worth, the [Wikipedia picture of a suffix tree](#) shows this pretty well. For example, if from the node for "NA" we follow the suffix edge, we get to the node for "A" and from there to "". We are always cutting off the leading character. So if we label state s with $\text{Key}(s)$, we have $f(\text{"NA"}) = \text{"A"}$ and $f(\text{"A"}) = \text{" "}$. (I don't know why he doesn't label states like this in the paper. It would simplify many explanations.)

Now this is very cool because we are computing only one column per viable prefix. But it's still expensive because we are inspecting each character and potentially traversing suffix edges for each one.

Algorithm B

Algorithm B's intent is to go faster by skipping through the input, touching only those characters likely to produce a new column, i.e. those that are the ends of input that match a previously unseen viable prefix of the pattern.

As you'd suspect, the key to the algorithm is the `loc` function. Roughly speaking, this will tell where the next "likely" input character is. The algorithm is quite a bit like A^* search. We maintain a min heap (which must have a delete operation) corresponding to the set S_i in the paper. (He calls it a dictionary, but this is not a very conventional use of the term.) The min heap contains potential "next states" keyed on the position of the next "likely character" as described above. Processing one character produces new entries. We keep going until the heap is empty.

You're absolutely right that here he gets sketchy. The theorems and lemmas are not tied together to make an argument on correctness. He assumes you will redo his work. I'm not entirely convinced by this hand-waving. But there does seem to be enough there to "decode" the algorithm he has in mind.

Another core concept is the set S_i and in particular the subset that remains not *eliminated*. We'll keep these un-eliminated states in the min-heap H .

You're right to say that the notation obscures the intent of S_i . As we process the input left-to-right and reach position i , we have amassed a set of viable prefixes seen so far. Each time a new one is found, a fresh dp column is computed. In the author's notation these prefixes would be Q_h for all $h \leq i$ or more formally $\{Q_h \mid h \leq i\}$. Each of these has a path from the root to a unique node. The set S_i consists of all the states we get by taking *one more step* from all these nodes along go-to edges in the trie. This produces the same result as going through the whole text looking for each occurrence of Q_h and the next character a , then adding the state corresponding to Q_h into S_i , but it's faster. The Keys for the S_i states are exactly the right candidates for the next viable prefix Q_{i+1} .

How do we choose the right candidate? Pick the one that occurs next after position i in the input. This is where `loc(s)` comes in. The `loc` value for a state s is just what I just said above: the position in the input starting at i where the viable prefix associated with that state occurs next.

The important point is that we don't want to just assign the newly found (by pulling the min `loc` value from H) "next" viable prefix as Q_{i+1} (the viable prefix for dp column $i+1$) and go on to the next character ($i+2$). This is where we must set the stage to skip ahead as far as possible to the *last* character k (with dp column k) such $Q_k = Q_{i+1}$. We skip ahead by following suffix edges

as in Algorithm A. Only this time we record our steps for future use by altering H: removing elements, which is the same as eliminating elements from S_i , and modifying loc values.

The definition of function $\text{loc}(s)$ is bare, and he never says how to compute it. Also unmentioned is that $\text{loc}(s)$ is also a function of i , the current input position being processed (that he jumps from j in earlier parts of the paper to i here for the current input position is unhelpful.) The impact is that $\text{loc}(s)$ *changes* as input processing proceeds.

It turns out that the part of the definition that applies to eliminated states "just happens" because states are marked eliminated upon removal from H. So for this case we need only check for a mark.

The other case - un-eliminated states - requires that we search forward in the input looking for the next occurrence in the text that is not covered by some other string. This notion of covering is to ensure we are always dealing with only "longest possible" viable prefixes. Shorter ones must be ignored to avoid outputting other than maximal matches. Now, searching forward sounds expensive, but happily we have a suffix trie already constructed, which allows us to do it in $O(|\text{Key}(s)|)$ time. The trie will have to be carefully annotated to return the relevant input position and to avoid covered occurrences of $\text{Key}(s)$, but it wouldn't be too hard. He never mentions what to do when the search fails. Here $\text{loc}(s) = \infty$, i.e. it's eliminated and should be deleted from H.

Perhaps the hairiest part of the algorithm is updating H to deal with cases where we add a new state s for a viable prefix that covers $\text{Key}(w)$ for some w that was already in H. This means we have to surgically update the $(\text{loc}(w) \Rightarrow w)$ element in H. It turns out the suffix trie yet again supports this efficiently with its suffix edges.

With all this in our heads, let's try for pseudocode.

```
H = { (0 => root) } // we use (loc => state) for min heap elements
until H is empty
  (j => s_j) = H.delete_min // remove the min loc mapping from
  (d, l) = dp(d', l', j) where (d', l') are cached at parent(s_j)
  Compute |Q_j| = l[h], h = argmax(i).d[i]<=k as in the paper
  r = s_j
  while depth(r) > |Q_j|
    mark r eliminated
    H.delete (r => r) // loc value doesn't matter
  end while
  set cached d, l on node r

// Add all the "next states" reachable from r by go-tos
for all s = g(r, a) for some character a
  unless s.eliminated?
    H.insert (loc(s) => s) // here is where we use the trie to find loc

    // Update H elements that might be newly covered
    w = f(s) // suffix transition
    while w != null
      unless w.eliminated?
        H.increase_key(loc(w) => w) // using explanation in Lemma 9.
        w = f(w) // suffix transition
      end unless
    end while
  end unless
end for
end until
```

Again I've omitted the output for simplicity. I will not say this is correct, but it's in the ballpark. One thing is that he mentions we should only process Q_j for nodes not before "visited," but I don't understand what "visited" means in this context. I think visited states by Algorithm A's definition won't occur because they've been removed from H. It's a puzzle...

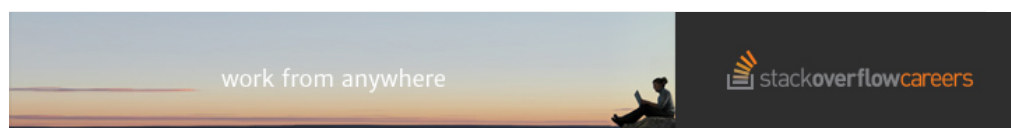
The `increase_key` operation in Lemma 9 is hastily described with no proof. His claim that the min operation is possible in $O(\log |\text{alphabet}|)$ time is leaving a lot to the imagination.

The number of quirks leads me to wonder if this is not the final draft of the paper. It is also a Springer publication, and this copy on-line would probably violate copyright restrictions if it were precisely the same. It might be worth looking in a library or paying for the final version to see if some of the rough edges were knocked off during final review.

This is as far as I can get. If you have specific questions, I'll try to clarify.

edited Oct 29 '13 at 16:26

community wiki
9 revs
Gene



This was the original question that started this thread.

Professor Esko Ukkonen published a [paper](#): *Approximate string-matching over suffix trees*. He

discusses 3 different algorithms that have different matching times:

1. Algorithm A: $O(mq + n)$
2. Algorithm B: $O(mq \log(q) + \text{size of the output})$
3. Algorithm C: $O(m^2q + \text{size of the output})$

Where m is the length of the substring, n is the length of the search string, and q is the edit distance.

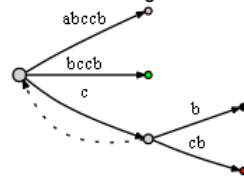
I've been trying to understand algorithm B but I'm having trouble following the steps. Does anyone have experience with this algorithm? An example or pseudo algorithm would be greatly appreciated.

In particular:

1. What does *size of the output* refer to in terms of the suffix tree or input strings? *The final output phase lists all occurrences of $\text{Key}(r)$ in T , for all states r marked for output.*
2. Looking at *Algorithm C*, the function dp is defined (page four); I don't understand what index i represents. It isn't initialized and doesn't appear to increment.

Here's what I believe (I stand to be corrected):

1. On page seven, we're introduced to suffix tree concepts; a state is effectively a node in the suffix tree: *let $root$ denote the initial state.*
2. $g(a, c) = b$ where a and b are nodes in the tree and c is a character or substring in the tree. So this represents a transition; from a , following the edges represented by c , we move to node b . This is referred to as the *go-to transition*. So for the suffix tree below,



$g(\text{root}, 'ccb') = \text{red node}$

3. $\text{Key}(a) = \text{edge sequence}$ where a represents a node in the tree. For example, $\text{Key}(\text{red node}) = 'ccb'$. So $g(\text{root}, \text{Key}(\text{red node})) = \text{red node}$.
4. $\text{Keys}(\text{Subset of node } S) = \{ \text{Key}(\text{node}) \mid \text{node} \in S \}$
5. There is a suffix function for nodes a and b , $f(a) = b$: for all (or perhaps there may exist) $a \neq \text{root}$, there exists a character c , a substring x , and a node b such that $g(\text{root}, cx) = a$ and $g(\text{root}, x) = b$. I think that this means, for the suffix tree example above, that $f(\text{pink node}) = \text{green node}$ where $c = 'a'$ and $x = 'bccb'$.
6. There is a mapping H that contains a node from the suffix tree and a value pair. The value is given by $\text{Loc}(w)$; I'm still uncertain how to evaluate the function. This dictionary contains nodes that have not been eliminated.
7. $\text{extract-min}(H)$ refers to attaining the entry with the smallest value in the pair $(\text{node}, \text{Loc}(w))$ from H .

The crux of the algorithm seems to be related to how $\text{Loc}(w)$ is evaluated. I've constructed my suffix tree using the combined answer [here](#); however, the algorithms work on a suffix trie (uncompressed suffix tree). Therefore concepts like the depth need to be maintained and processed differently. In the suffix trie the depth would represent the suffix length; in a suffix tree, the depth would simply represent the node depth in the tree.

edited Oct 26 '13 at 11:59

community wiki
2 revs
Pooven